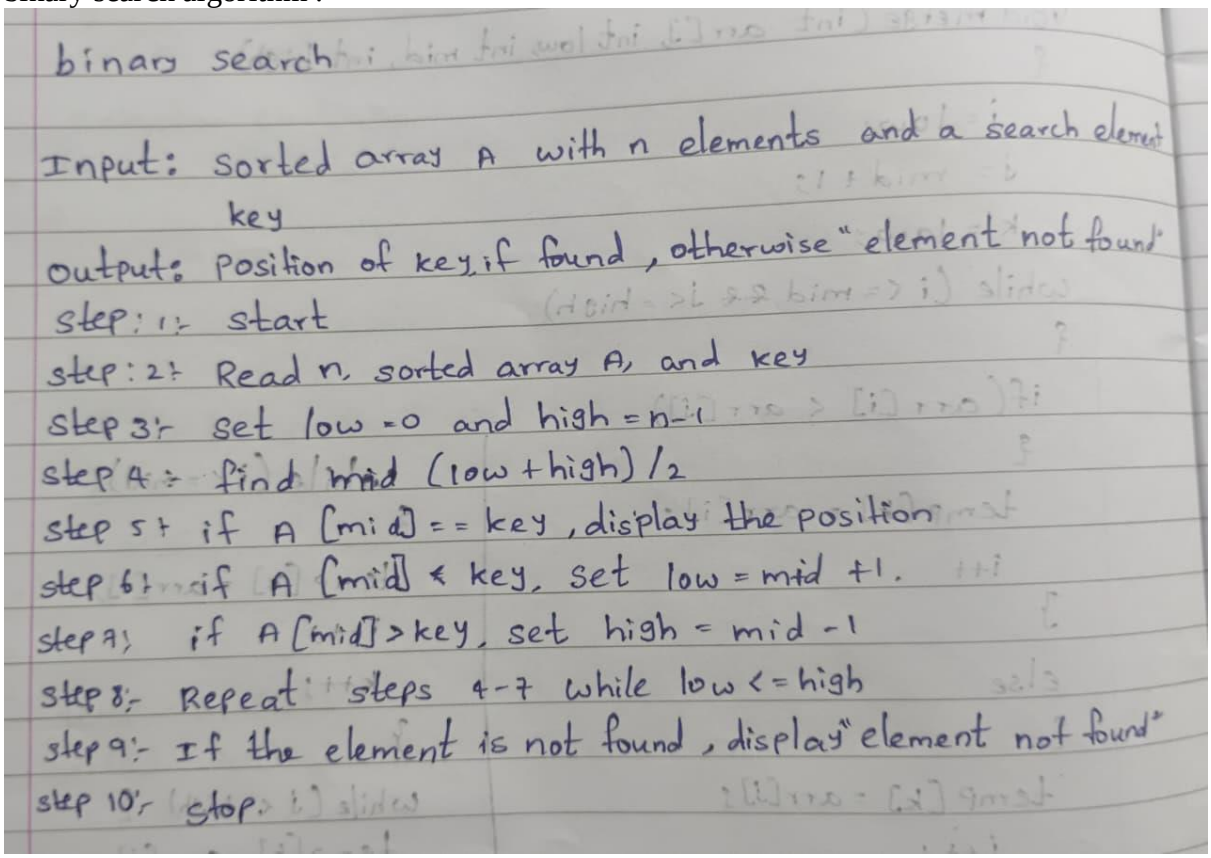
PRACTICAL - 2

AIM :Implementation and Time analysis of linear and binary search algorithm

binary search algorithm :



code :

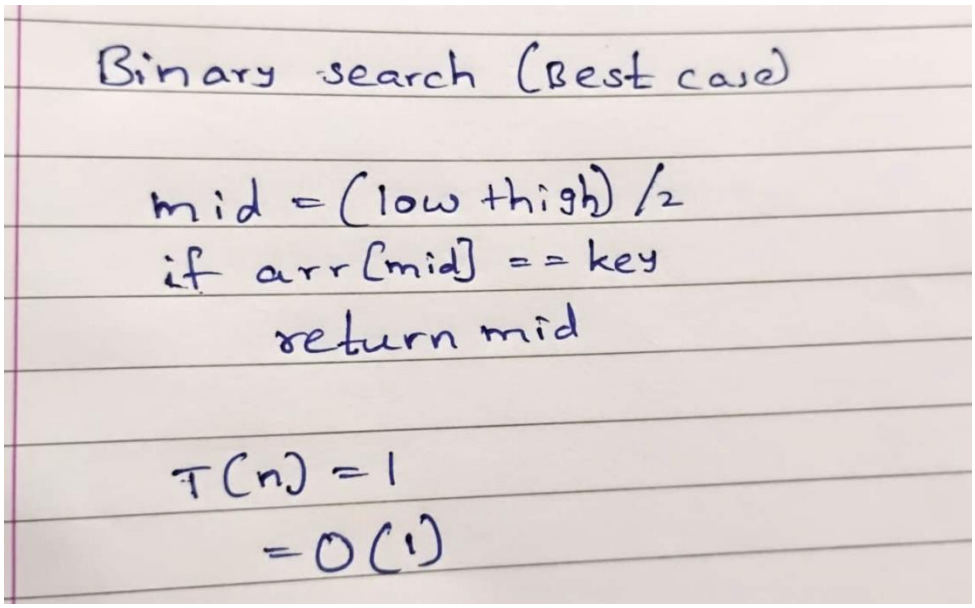
```
#include <iostream>
using namespace std;

int main()
{
   int n, arr[100], key;
   int low, high, mid;

   cout << "Enter size of array: ";
   cin >> n;
```

```
    cout << "Enter sorted array elements: ";
    for(int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Enter element to search: ";
    cin >> key;

    low = 0;
    high = n - 1;

    while(low <= high)
    {
        mid = (low + high) / 2;

        if(arr[mid] == key)
        {
            cout << "Element found at position " << mid + 1;
            return 0;
        }
        else if(arr[mid] < key)
        {
            low = mid + 1;
        }
        else
        {
            high = mid - 1;
        }
    }

    cout << "Element not found";

    return 0;
}
```

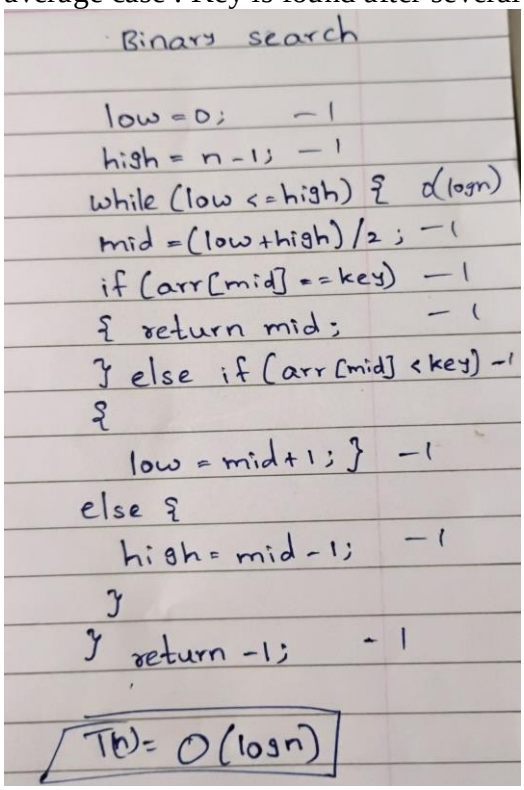
Output

```
Enter size of array: 5
Enter sorted array elements: 2
5
7
35
89
Enter element to search: 5
Element found at position 2
```

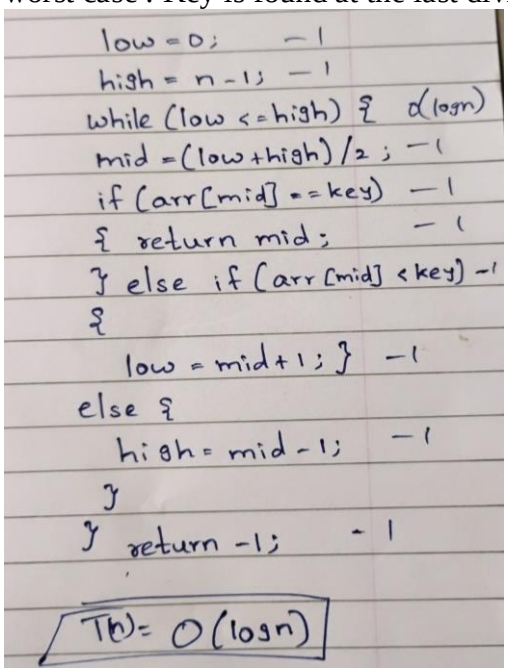
time complexity : Step Count / Frequency Count Method)
best case :



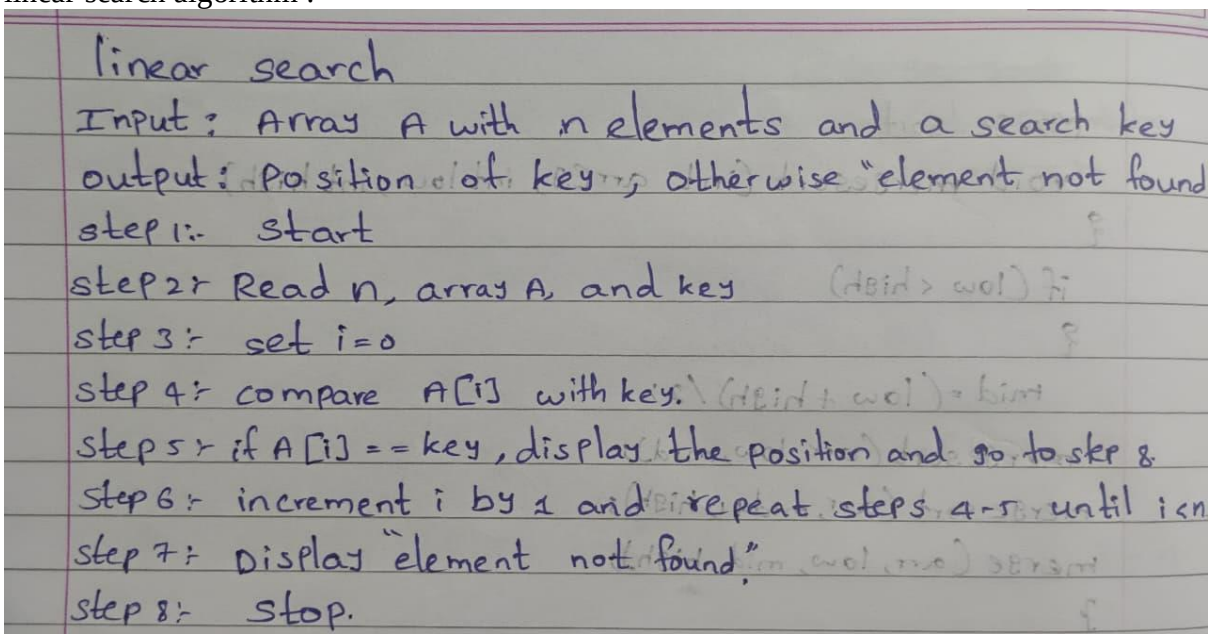
average case : Key is found after several divisions



worst case : Key is found at the last division / not found



linear search algorithm :



code :

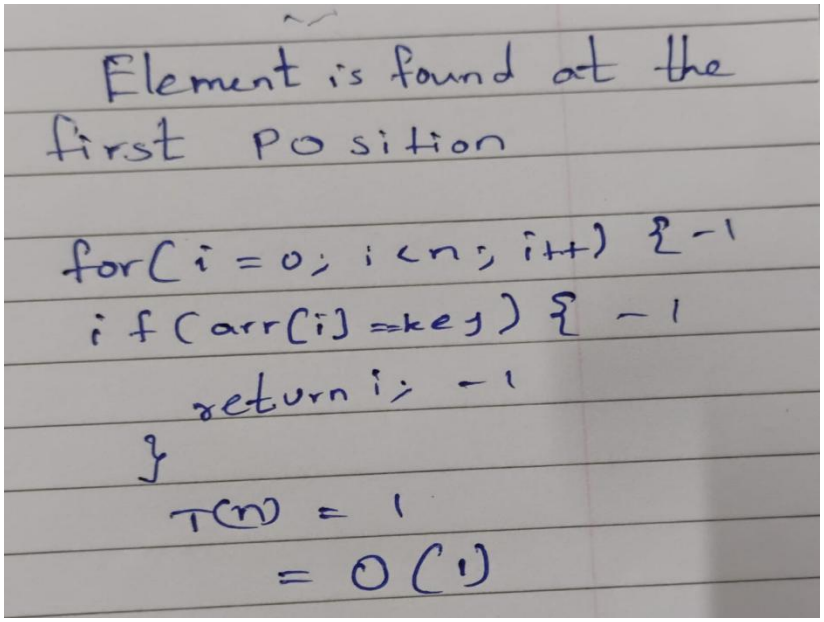
```
#include <iostream>
using namespace std;

int main()
{
   int n, arr[100], key;

   cout << "Enter size of array: ";
   cin >> n;

   cout << "Enter array elements: ";
   for(int i = 0; i < n; i++)
      cin >> arr[i];

   cout << "Enter element to search: ";
   cin >> key;

   for(int i = 0; i < n; i++)
   {
      if(arr[i] == key)
      {
         cout << "Element found at position " << i + 1;
```

```
            return 0;
        }
    }

    cout << "Element not found";

    return 0;
}
```

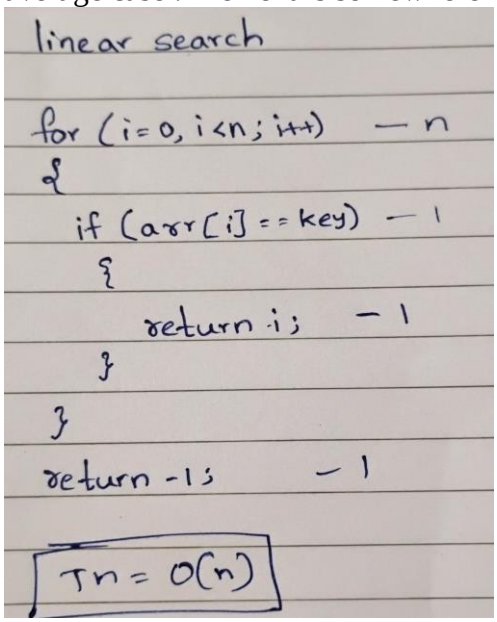
Output

```
Enter size of array: 5
Enter array elements: 34
67
23
65
1
Enter element to search: 24
Element not found
```

time complexity (Step Count / Frequency Count Method)
best case :



average case : Element is somewhere in the middle



worst case : Element is at the last position / not found